

Celebal Summer Internship 2026 – SQL Week 3

Project: Advanced SQL Data Analysis

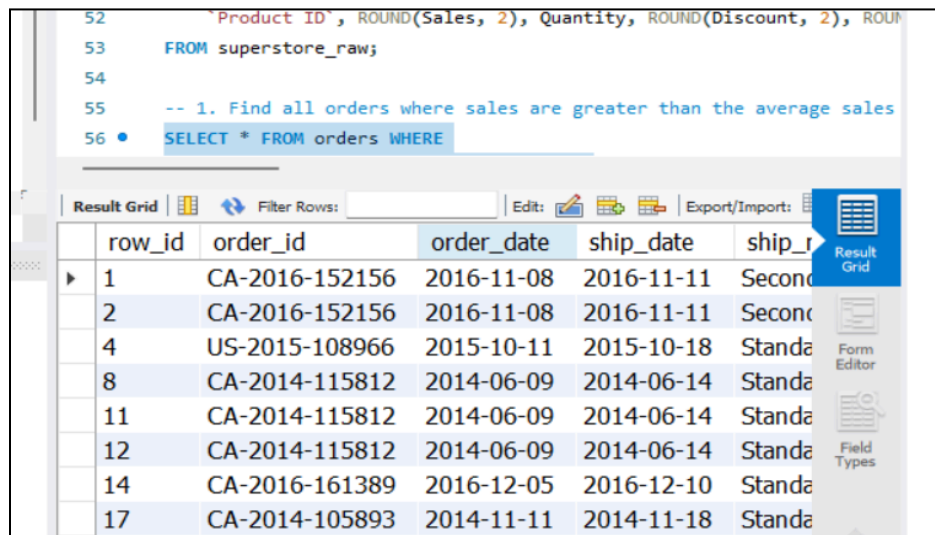
Internship: Celebal Summer Internship 2026 – SQL Week 3

Author: Manjit_Bajaj

Purpose

This document contains screenshots of the advanced SQL queries executed during the analysis along with their corresponding outputs. These screenshots serve as evidence of successful query execution and support the findings presented in the project.

1. Find all orders where sales are greater than the average sales. *(Subquery)*



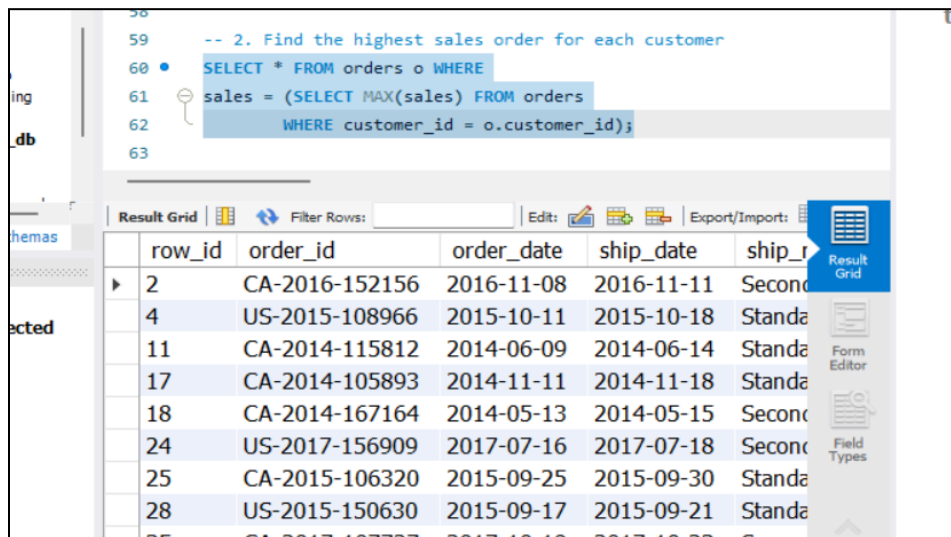
The screenshot shows a SQL query editor with the following code:

```
52 'Product ID', ROUND(Sales, 2), Quantity, ROUND(Discount, 2), ROUND
53 FROM superstore_raw;
54
55 -- 1. Find all orders where sales are greater than the average sales
56 • SELECT * FROM orders WHERE
```

Below the editor is a 'Result Grid' showing the output of the query. The grid has columns: row_id, order_id, order_date, ship_date, and ship_r. The data is as follows:

row_id	order_id	order_date	ship_date	ship_r
1	CA-2016-152156	2016-11-08	2016-11-11	Second
2	CA-2016-152156	2016-11-08	2016-11-11	Second
4	US-2015-108966	2015-10-11	2015-10-18	Standard
8	CA-2014-115812	2014-06-09	2014-06-14	Standard
11	CA-2014-115812	2014-06-09	2014-06-14	Standard
12	CA-2014-115812	2014-06-09	2014-06-14	Standard
14	CA-2016-161389	2016-12-05	2016-12-10	Standard
17	CA-2014-105893	2014-11-11	2014-11-18	Standard

2. Find the highest sales order for each customer. *(Subquery)*



The screenshot shows a SQL query editor with the following code:

```
59 -- 2. Find the highest sales order for each customer
60 • SELECT * FROM orders o WHERE
61 sales = (SELECT MAX(sales) FROM orders
62 WHERE customer_id = o.customer_id);
63
```

Below the editor is a 'Result Grid' showing the output of the query. The grid has columns: row_id, order_id, order_date, ship_date, and ship_r. The data is as follows:

row_id	order_id	order_date	ship_date	ship_r
2	CA-2016-152156	2016-11-08	2016-11-11	Second
4	US-2015-108966	2015-10-11	2015-10-18	Standard
11	CA-2014-115812	2014-06-09	2014-06-14	Standard
17	CA-2014-105893	2014-11-11	2014-11-18	Standard
18	CA-2014-167164	2014-05-13	2014-05-15	Second
24	US-2017-156909	2017-07-16	2017-07-18	Second
25	CA-2015-106320	2015-09-25	2015-09-30	Standard
28	US-2015-150630	2015-09-17	2015-09-21	Standard
29	CA-2017-107727	2017-10-10	2017-10-22	Second

3. Calculate total sales for each customer. (CTE)

```
61 Sales = (SELECT top(Sales) FROM Orders
62         WHERE customer_id = o.customer_id);
63
64 -- 3. Calculate total sales for each customer.
65 • WITH customer_sales AS
```

customer_id	total_sales
CG-12520	993.90
DV-13045	203.17
SO-20335	1774.52
BH-11710	4844.59
AA-10480	624.89
IM-15070	4115.60
HP-14815	1593.58
PK-19075	1424.15
AG-10270	55.50

4. Find customers whose total sales are above average. (CTE + Subquery)

```
69 SELECT * FROM customer_sales;
70
71 -- 4. Find customers whose total sales are above average.
72 • WITH customer_sales as (
```

customer_id	total_sales
SO-20335	1774.52
BH-11710	4844.59
IM-15070	4115.60
HP-14815	1593.58
PK-19075	1424.15
EB-13870	1790.06
TB-21520	4320.10
GH-14485	1288.46

5. Rank all customers based on total sales. (Window Function)

```
77
78 -- 5. Rank all customers based on total sales.
79 • WITH customer_sales as
```

customer_id	total_sales	sales_rank
SM-20320	24195.66	1
TA-21385	13716.46	2
KL-16645	12162.70	3
BM-11140	11037.14	4
GT-14635	9351.20	5
KD-16270	7956.66	6
SB-20290	7688.06	7
JM-15865	6749.13	8

6. Assign row numbers to each order within a customer. (*Window Function + PARTITION BY*)

```

83
84 -- 6. Assign row numbers to each order within a customer.
85 SELECT order_id, customer_id, sales, ROW_NUMBER() OVER(

```

order_id	customer_id	sales	row_no
CA-2014-128055	AA-10315	673.57	1
CA-2017-147039	AA-10315	362.94	2
CA-2014-128055	AA-10315	52.98	3
CA-2015-121391	AA-10315	26.96	4
CA-2017-147039	AA-10315	11.54	5
CA-2016-131065	AA-10375	499.98	1
CA-2015-140921	AA-10375	149.97	2
CA-2014-130729	AA-10375	34.27	3

7. Display top 3 customers based on total sales. (*Window Function*)

```

110
111 -- 7. Display top 3 customers based on total sales.
112 WITH customer_sales as (SELECT customer_id, sum(sales) as total_sales
113 FROM orders GROUP BY customer_id),
114 Ranking as ( SELECT *, RANK() OVER(ORDER BY total_sales DESC) ranking
115 Select * FROM ranking WHERE ranking<=3;
116 /*
117 Insight: Displays the three highest revenue-generating customers.
118 These customers contribute significantly to overall sales and are val

```

customer_name	total_sales	sales_rank
Sean Miller	24195.66	1
Tom Ashbrook	13716.46	2
Ken Lonsdale	12162.70	3
Becky Martin	11037.14	4
Grant Thornton	9351.20	5
Karen Daniels	7956.66	6
Sean Braxton	7688.06	7
John Murray	6749.13	8

Final Combined Query

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

(Use **JOIN** + **CTE** + **Window Function** together)

The screenshot shows a SQL query editor window titled "Query 1". The query is as follows:

```
92 Select * FROM ranking WHERE ranking<=3;
93
94 -- Final combined query
95 with customer_sales as (
96 SELECT customer_id, sum(sales) as total_sales FROM orders GROUP BY customer_id
97
98 SELECT c.customer_name, cs.total_sales, RANK() OVER(ORDER BY cs.total_sales) as sales_rank
99 FROM customer_sales cs
100 JOIN customers c
101 ON cs.customer_id = c.customer_id ORDER BY sales_rank;
```

Below the query editor is a "Result Grid" showing the output of the query. The grid has four columns: customer_name, total_sales, and sales_rank. The data is as follows:

customer_name	total_sales	sales_rank
Sean Miller	24195.66	1
Tom Ashbrook	13716.46	2
Ken Lonsdale	12162.70	3
Becky Martin	11037.14	4
Grant Thornton	9351.20	5
Karen Daniels	7956.66	6
Sean Braxton	7688.06	7
John Murray	6749.13	8

Conclusion: This project successfully demonstrated the use of advanced SQL concepts, including Subqueries, Common Table Expressions (CTEs), and Window Functions, to analyze the Superstore sales dataset. The queries were used to extract meaningful business information such as above-average sales, customer-wise sales performance, customer rankings, and order-level analysis.

The analysis highlighted how SQL can be used to transform raw transactional data into valuable business insights. By combining JOINS, CTEs, and Window Functions, comprehensive customer sales reports and rankings were generated, enabling better understanding of customer behavior and sales distribution. Overall, this project strengthened practical SQL skills and showcased the application of advanced querying techniques in real-world business analytics.